

Inżynieria danych

Wykład 10: transakcje (cz. II)

Katarzyna Grygiel

Semestr letni 2025/2026

Blokady w służbie bezpieczeństwa: idea

plan

T_1	T_2
begin	
lock (A)	
r(A)	begin
	lock (A)
w(A)	
r(A)	
unlock (A)	
	r(A)
	w(A)
commit	unlock (A)
	commit

menadżer blokad

przyznanie $T_1 \rightarrow A$

odmowa

zwolnienie blokady

przyznanie $T_2 \rightarrow A$

zwolnienie blokady

Blokady i ich rodzaje

Blokada (*lock*) to zmienna skojarzona z pewną daną w bazie, która określa możliwość wykonania ustalonej operacji na tej danej.

Blokady stanowią podstawowy mechanizm zapobiegania konfliktom przy współbieżnym wykonywaniu transakcji.

Blokady i ich rodzaje

Blokada (*lock*) to zmienna skojarzona z pewną daną w bazie, która określa możliwość wykonania ustalonej operacji na tej danej.

Blokady stanowią podstawowy mechanizm zapobiegania konfliktom przy współbieżnym wykonywaniu transakcji.

Blokada przyjmuje jedną z trzech wartości:

- 0 – dana nie jest zablokowana,

Blokady i ich rodzaje

Blokada (*lock*) to zmienna skojarzona z pewną daną w bazie, która określa możliwość wykonania ustalonej operacji na tej danej.

Blokady stanowią podstawowy mechanizm zapobiegania konfliktom przy współbieżnym wykonywaniu transakcji.

Blokada przyjmuje jedną z trzech wartości:

- 0 – dana nie jest zablokowana,
- S – blokada **dzielona** (*shared lock*): transakcja otrzymuje współdzielony dostęp do żądanego zasobu, np. kilka transakcji może jednocześnie dokonywać odczytu tej samej krotki,

Blokady i ich rodzaje

Blokada (*lock*) to zmienna skojarzona z pewną daną w bazie, która określa możliwość wykonania ustalonej operacji na tej danej.

Blokady stanowią podstawowy mechanizm zapobiegania konfliktom przy współbieżnym wykonywaniu transakcji.

Blokada przyjmuje jedną z trzech wartości:

- 0 – dana nie jest zablokowana,
- S – blokada **dzielona** (*shared lock*): transakcja otrzymuje współdzielony dostęp do żądanego zasobu, np. kilka transakcji może jednocześnie dokonywać odczytu tej samej krotki,
- X – blokada **wyłączna** (*exclusive lock*): transakcja otrzymuje wyłączność na wprowadzenie zmian w zablokowanym zasobie, np. tylko jedna transakcja może modyfikować zawartość krotki.

Zarządzanie blokadami

Jeśli transakcja ma blokadę X dla pewnego obiektu, to żadna inna transakcja nie może założyć żadnej blokady na tym obiekcie.

Jeśli transakcja ma blokadę S dla pewnego obiektu, to żadna inna transakcja nie może założyć blokady X na tym obiekcie.

Zarządzanie blokadami

Jeśli transakcja ma blokadę X dla pewnego obiektu, to żadna inna transakcja nie może założyć żadnej blokady na tym obiekcie.

Jeśli transakcja ma blokadę S dla pewnego obiektu, to żadna inna transakcja nie może założyć blokady X na tym obiekcie.

Macierz kompatybilności:

	dzielona	wyłączna
dzielona	✓	✗
wyłączna	✗	✗

Zarządzanie blokadami

Jeśli transakcja ma blokadę X dla pewnego obiektu, to żadna inna transakcja nie może założyć żadnej blokady na tym obiekcie.

Jeśli transakcja ma blokadę S dla pewnego obiektu, to żadna inna transakcja nie może założyć blokady X na tym obiekcie.

Macierz kompatybilności:

	dzielona	wyłączna
dzielona	✓	✗
wyłączna	✗	✗

Transakcja wysyła żądanie przyznania blokady do **menadżera blokad**, który przyznaje blokady lub ich odmawia.

Gdy transakcja zwalnia blokadę, to odpowiednia informacja jest aktualizowana przez menadżer blokad.

Blokady: przykład

plan

T_1	T_2
begin	
x-lock(A)	
r(A)	
w(A)	
unlock(A)	begin
	x-lock(A)
	w(A)
	unlock(A)
s-lock(A)	
r(A)	
unlock(A)	
commit	commit

menadżer blokad

przyznanie $T_1 \rightarrow X(A)$

zwolnienie blokady
przyznanie $T_2 \rightarrow X(A)$

zwolnienie blokady
przyznanie $T_1 \rightarrow S(A)$

zwolnienie blokady

Blokady: przykład

plan

T_1	T_2
begin	
x-lock(A)	
r(A)	
w(A)	
unlock(A)	begin
	x-lock(A)
	w(A)
	unlock(A)
s-lock(A)	
r(A)	
unlock(A)	
commit	commit

menadżer blokad

przyznanie $T_1 \rightarrow X(A)$

zwolnienie blokady
przyznanie $T_2 \rightarrow X(A)$

zwolnienie blokady
przyznanie $T_1 \rightarrow S(A)$

zwolnienie blokady

No dobrze, ale tu też mamy problem. W grafie konfliktów jest cykl i mamy anomalię niepowtarzalnego odczytu.

Protokół blokowania dwufazowego

Protokół **blokowania dwufazowego** (*two-phase locking, 2PL*) jest algorytmem kontroli współbieżnego dostępu, sprawdzającym na bieżąco, czy transakcja może otrzymać dostęp do obiektu bazodanowego.

Protokół blokowania dwufazowego

Protokół **blokowania dwufazowego** (*two-phase locking, 2PL*) jest algorytmem kontroli współbieżnego dostępu, sprawdzającym na bieżąco, czy transakcja może otrzymać dostęp do obiektu bazodanowego.

Protokół ten nie musi znać a priori wszystkich zapytań, z których składa się transakcja.

Protokół blokowania dwufazowego

Protokół **blokowania dwufazowego** (*two-phase locking, 2PL*) jest algorytmem kontroli współbieżnego dostępu, sprawdzającym na bieżąco, czy transakcja może otrzymać dostęp do obiektu bazodanowego.

Protokół ten nie musi znać a priori wszystkich zapytań, z których składa się transakcja.

Jak sama nazwa wskazuje, 2PL składa się z dwóch faz. Są to:

- **faza wzrostu** (*growing phase*), podczas której każda z transakcji przesyła swoje zapotrzebowania na blokady do menadżera blokad, który je w tej fazie przyznaje lub ich odmawia,
- **faza zmniejszania** (*shrinking phase*), podczas której transakcje mogą tylko zwalniać przyznane im wcześniej blokady; w tej fazie nie mogą być przyznane nowe blokady.

Protokół blokowania dwufazowego rysunkowo



Protokół blokowania dwufazowego rysunkowo



Protokół blokowania dwufazowego: przykład

plan

T_1	T_2
begin	
x-lock(A)	
r(A)	
w(A)	
	begin
	x-lock(A)
r(A)	
unlock(A)	
commit	
	w(A)
	unlock(A)
	commit

Protokół blokowania dwufazowego: przykład

plan

T_1	T_2
begin	
x-lock (A)	
r(A)	
w(A)	
	begin
	x-lock (A)
r(A)	
unlock (A)	
commit	
	w(A)
	unlock (A)
	commit

menadżer blokad

przyznanie $T_1 \rightarrow X(A)$
odmowa $T_2 \rightarrow X(A)$
zwolnienie blokady
przyznanie $T_2 \rightarrow X(A)$
zwolnienie blokady

Protokół blokowania dwufazowego: przykład

plan

T_1	T_2
begin	
x-lock(A)	
r(A)	
w(A)	
	begin
	x-lock(A)
r(A)	
unlock(A)	
commit	
	w(A)
	unlock(A)
	commit

menadżer blokad

przyznanie $T_1 \rightarrow X(A)$
odmowa $T_2 \rightarrow X(A)$
zwolnienie blokady
przyznanie $T_2 \rightarrow X(A)$
zwolnienie blokady

Powyższa realizacja jest szeregowalna dzięki 2PL.

Kaskadowe wycofania

Algorytm dwufazowego blokowania zapewnia konfliktową szeregowalność. Jest jednak podatny na kaskadowe wycofania.

Kaskadowe wycofania

Algorytm dwufazowego blokowania zapewnia konfliktową szeregowalność. Jest jednak podatny na kaskadowe wycofania.

plan

T_1	T_2
begin	begin
x-lock(A)	
x-lock(B)	
r(A)	
w(A)	
unlock(A)	
	x-lock(A)
	r(A)
	w(A)
r(B)	
w(B)	
rollback	

Kaskadowe wycofania

Algorytm dwufazowego blokowania zapewnia konfliktową szeregowalność. Jest jednak podatny na kaskadowe wycofania.

plan

T_1	T_2
begin	begin
x-lock(A)	
x-lock(B)	
r(A)	
w(A)	
unlock(A)	
	x-lock(A)
	r(A)
	w(A)
r(B)	
w(B)	
rollback	

Realizacja po lewej jest dopuszczalna przez 2PL, ale po wycofaniu T_1 transakcja T_2 też musi być wycofana.

Kaskadowe wycofania

Algorytm dwufazowego blokowania zapewnia konfliktową szeregowalność. Jest jednak podatny na kaskadowe wycofania.

plan

T_1	T_2
begin	begin
x-lock(A)	
x-lock(B)	
r(A)	
w(A)	
unlock(A)	
	x-lock(A)
	r(A)
	w(A)
r(B)	
w(B)	
rollback	

Realizacja po lewej jest dopuszczalna przez 2PL, ale po wycofaniu T_1 transakcja T_2 też musi być wycofana.

Powiemy, że realizacja **unik**a kaskadowych **wycofań**, jeżeli czyta zmiany dokonane tylko przez zatwierdzone transakcje.

Kaskadowe wycofania

Algorytm dwufazowego blokowania zapewnia konfliktową szeregowalność. Jest jednak podatny na kaskadowe wycofania.

plan

T_1	T_2
begin	begin
x-lock(A)	
x-lock(B)	
r(A)	
w(A)	
unlock(A)	
	x-lock(A)
	r(A)
	w(A)
r(B)	
w(B)	
rollback	

Realizacja po lewej jest dopuszczalna przez 2PL, ale po wycofaniu T_1 transakcja T_2 też musi być wycofana.

Powiemy, że realizacja **unik**a kaskadowych **wycofań**, jeżeli czyta zmiany dokonane tylko przez zatwierdzone transakcje.

Dzięki takiej własności realizacja będzie wolna od anomalii brudnego odczytu.

Zalety, wady i modyfikacja 2PL

Algorytm dwufazowego odczytu zapewnia, że nie pojawi się anomalia niepowtarzalnego odczytu.

Zalety, wady i modyfikacja 2PL

Algorytm dwufazowego odczytu zapewnia, że nie pojawi się anomalia niepowtarzalnego odczytu.

Ma jednak pewne wady:

- może dopuścić do brudnych odczytów,
- może prowadzić do zakleszczeń,
- może nie dopuścić do wykonania realizacji, która jest poprawna.

Zalety, wady i modyfikacja 2PL

Algorytm dwufazowego odczytu zapewnia, że nie pojawi się anomalia niepowtarzalnego odczytu.

Ma jednak pewne wady:

- może dopuścić do brudnych odczytów,
- może prowadzić do zakleszczeń,
- może nie dopuścić do wykonania realizacji, która jest poprawna.

Rozwiązaniem pierwszego z problemów jest zastosowanie zmodyfikowanej wersji 2PL: protokołu **ściślego blokowania dwufazowego** (*strict 2PL*), który nie pozwala na zwalnianie blokad przed zatwierdzeniem lub wycofaniem transakcji.

Ścisłe realizacje

Realizację nazwiemy **ścisłą**, jeżeli dla dowolnych dwóch operacji $w_i(A) \in T_i$ oraz $o_j(A) \in T_j$ takich, że $w_i(A) \rightarrow o_j(A)$ oraz $i \neq j$ zatwierdzenie lub wycofanie transakcji T_i poprzedza operację $o_j(A)$.

Innymi słowy zapis dokonywany przez transakcję T nie jest odczytywany ani nadpisywany przez inną transakcję dopóki transakcja T się nie zakończy.

Ścisłe realizacje

Realizację nazwiemy **ściłą**, jeżeli dla dowolnych dwóch operacji $w_i(A) \in T_i$ oraz $o_j(A) \in T_j$ takich, że $w_i(A) \rightarrow o_j(A)$ oraz $i \neq j$ zatwierdzenie lub wycofanie transakcji T_i poprzedza operację $o_j(A)$.

Innymi słowy zapis dokonywany przez transakcję T nie jest odczytywany ani nadpisywany przez inną transakcję dopóki transakcja T się nie zakończy.

Jeżeli transakcja jest ścisła, to unika kaskadowych wycofań. Ta druga własność gwarantuje, że realizacja jest **odtwarzalna**, czyli zatwierdzenie każdej z jej transakcji T następuje dopiero po zatwierdzeniu innych transakcji, które wprowadzały zmiany odczytywane przez T .

Przykład bez dwufazowego blokowania

T_1	T_2
begin	begin
x-lock(A)	
r(A)	
	s-lock(A)
$A = A - 100$	
unlock(A)	
	r(A)
	unlock(A)
	s-lock(B)
x-lock(B)	
	r(B)
	unlock(B)
r(B)	SELECT A + B
$B = B + 100$	
unlock(B)	
commit	commit

Początkowy stan:

$A = 1000, \quad B = 1000.$

Przykład bez dwufazowego blokowania

T_1	T_2
begin	begin
x-lock(A)	
r(A)	
	s-lock(A)
$A = A - 100$	
unlock(A)	
	r(A)
	unlock(A)
	s-lock(B)
x-lock(B)	
	r(B)
	unlock(B)
r(B)	SELECT $A + B$
$B = B + 100$	
unlock(B)	
commit	commit

Początkowy stan:

$$A = 1000, \quad B = 1000.$$

Transakcja T_2 zwraca

$$A + B = 1900.$$

Przykład z dwufazowym blokowaniem (2PL)

T_1	T_2
begin	begin
x-lock(A)	
r(A)	s-lock(A)
$A = A - 100$	
x-lock(B)	
unlock(A)	r(A)
	s-lock(B)
r(B)	
$B = B + 100$	
unlock(B)	r(B)
commit	unlock(A)
	unlock(B)
	SELECT $A + B$
	commit

Początkowy stan:

$A = 1000, \quad B = 1000.$

Przykład z dwufazowym blokowaniem (2PL)

T_1	T_2
begin	begin
x-lock (A)	
r(A)	s-lock (A)
$A = A - 100$	
x-lock (B)	
unlock (A)	r(A)
	s-lock (B)
r(B)	
$B = B + 100$	
unlock (B)	r(B)
commit	unlock (A)
	unlock (B)
	SELECT $A + B$
	commit

Początkowy stan:

$$A = 1000, \quad B = 1000.$$

Transakcja T_2 zwraca

$$A + B = 2000.$$

Przykład ze ścisłym dwufazowym blokowaniem

T_1	T_2
begin	begin
x-lock (A)	
r(A)	s-lock (A)
$A = A - 100$	
x-lock (B)	
r(B)	
$B = B + 100$	
unlock (A)	
unlock (B)	r(A)
commit	s-lock (B)
	r(B)
	SELECT $A + B$
	unlock (A)
	unlock (B)
	commit

Początkowy stan:

$A = 1000, \quad B = 1000.$

Przykład ze ścisłym dwufazowym blokowaniem

T_1	T_2
begin	begin
x-lock (A)	
r(A)	s-lock (A)
$A = A - 100$	
x-lock (B)	
r(B)	
$B = B + 100$	
unlock (A)	
unlock (B)	r(A)
commit	s-lock (B)
	r(B)
	SELECT $A + B$
	unlock (A)
	unlock (B)
	commit

Początkowy stan:

$$A = 1000, \quad B = 1000.$$

Transakcja T_2 zwraca

$$A + B = 2000.$$

Zakleszczenia

Stan wzajemnego oczekiwania transakcji na zwolnienie zasobów nazywamy **zakleszczeniem** transakcji.

Zakleszczenia

Stan wzajemnego oczekiwania transakcji na zwolnienie zasobów nazywamy **zakleszczeniem** transakcji.

T_1	T_2
begin	begin
x-lock(A)	
	s-lock(B)
	r(B)
	s-lock(A)
r(A)	
x-lock(B)	
⋮	⋮

Zakleszczenia

Stan wzajemnego oczekiwania transakcji na zwolnienie zasobów nazywamy **zakleszczeniem** transakcji.

T_1	T_2
begin	begin
x-lock(A)	
	s-lock(B)
	r(B)
	s-lock(A)
r(A)	
x-lock(B)	
⋮	⋮

W systemach baz danych stosowane są dwa podejścia do problemu zakleszczenia:

- wykrywanie i rozwiązywanie zakleszczeń,
- zapobieganie wystąpieniu zakleszczeń.

Zakleszczenia: graf oczekiwania

System bazodanowy tworzy **graf oczekiwania**, aby móc śledzić, która z transakcji oczekuje na zwolnienie blokady przez którą transakcję:

- wierzchołkami w grafie są transakcje,
- krawędź $T_i \rightarrow T_j$ istnieje wtedy, gdy T_i czeka na zwolnienie blokady przez T_j .

Zakleszczenia: graf oczekiwania

System bazodanowy tworzy **graf oczekiwania**, aby móc śledzić, która z transakcji oczekuje na zwolnienie blokady przez którą transakcję:

- wierzchołkami w grafie są transakcje,
- krawędź $T_i \rightarrow T_j$ istnieje wtedy, gdy T_i czeka na zwolnienie blokady przez T_j .

System co jakiś czas sprawdza, czy graf oczekiwania nie zawiera cyklu i wtedy podejmuje decyzję o jego przerwaniu.

Zakleszczenia są stosunkowo rzadkie, taniej jest je usuwać w momencie, gdy już się pojawiają.

Wykrywanie zakleszczeń: kozioł ofiarny

Gdy SZBD wykryje zakleszczenie, wybierze transakcję, która zostanie wycofana i przerwie cykl w grafie oczekiwania.

Wykorzystuje się kilka różnych metod wskazywania „kozła ofiarnego”: może to być najstarsza transakcja, najmłodsza, najmniej zaawansowana, zajmująca najwięcej blokad itp.

Wykrywanie zakleszczeń: kozioł ofiarny

Gdy SZBD wykryje zakleszczenie, wybierze transakcję, która zostanie wycofana i przerwie cykl w grafie oczekiwania.

Wykorzystuje się kilka różnych metod wskazywania „kozła ofiarnego”: może to być najstarsza transakcja, najmłodsza, najmniej zaawansowana, zajmująca najwięcej blokad itp.

Transakcja będąca kozłem ofiarnym albo zostaje wycofana, albo rozpocznie się na nowo (w zależności od ustawień). W drugim wypadku należy zwrócić uwagę na to, jak wiele razy była ona już wznowiana w przeszłości.

Wykrywanie zakleszczeń: kozioł ofiarny

Gdy SZBD wykryje zakleszczenie, wybierze transakcję, która zostanie wycofana i przerwie cykl w grafie oczekiwania.

Wykorzystuje się kilka różnych metod wskazywania „kozła ofiarnego”: może to być najstarsza transakcja, najmłodsza, najmniej zaawansowana, zajmująca najwięcej blokad itp.

Transakcja będąca kozłem ofiarnym albo zostaje wycofana, albo rozpocznie się na nowo (w zależności od ustawień). W drugim wypadku należy zwrócić uwagę na to, jak wiele razy była ona już wznawiana w przeszłości.

Im częściej system będzie sprawdzał, czy nie pojawiło się zakleszczenie, tym szybciej poradzi sobie z jego usunięciem, ale równocześnie tym większe będzie obciążenie systemu.

Unikanie zakleszczeń

Drugą z metod radzenia sobie z zakleszczeniami jest przypisanie transakcjom priorytetów opartych na znacznikach czasowych. Wyższy priorytet otrzymuje transakcja posiadająca starszy (mniejszy) znacznik.

Unikanie zakleszczeń

Drugą z metod radzenia sobie z zakleszczeniami jest przypisanie transakcjom priorytetów opartych na znacznikach czasowych. Wyższy priorytet otrzymuje transakcja posiadająca starszy (mniejszy) znacznik.

Rozważmy sytuację, gdy transakcja T_1 wysyła żądanie blokady, która jest już przypisana transakcji T_2 .

Schemat **starsza czeka na młodszą** (*wait-die*):

- jeżeli T_1 ma wyższy priorytet niż T_2 , to T_1 czeka na zakończenie T_2 ,
- jeżeli T_1 ma niższy priorytet niż T_2 , to następuje wycofanie T_1 ;

Schemat **młodsza czeka na starszą** (*wound-wait*):

- jeżeli T_1 ma wyższy priorytet niż T_2 , to następuje wycofanie T_2 i T_1 otrzymuje żadaną blokadę,
- jeżeli T_1 ma niższy priorytet niż T_2 , to T_1 czeka na zakończenie T_2 .

Schematy czekania/wycofywania na przykładzie

Rozważmy transakcje T_1 i T_2 takie, że T_1 ma wyższy priorytet niż T_2 .

T_1	T_2
begin	
	begin
	x-lock(A)
x-lock(A)	

Schematy czekania/wycofywania na przykładzie

Rozważmy transakcje T_1 i T_2 takie, że T_1 ma wyższy priorytet niż T_2 .

T_1	T_2
begin	
	begin
	x-lock(A)
x-lock(A)	

schemat wait-die:

$\rightsquigarrow T_1$ czeka.

schemat wound-wait:

$\rightsquigarrow T_2$ zostaje wycofana.

Schematy czekania/wycofywania na przykładzie

Rozważmy transakcje T_1 i T_2 takie, że T_1 ma wyższy priorytet niż T_2 .

T_1	T_2
begin	
	begin
	x-lock(A)
x-lock(A)	

schemat wait-die:

$\rightsquigarrow T_1$ czeka.

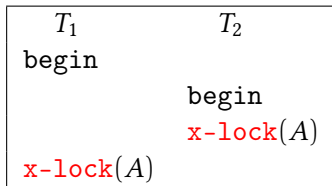
schemat wound-wait:

$\rightsquigarrow T_2$ zostaje wycofana.

T_1	T_2
begin	
	begin
x-lock(A)	
	x-lock(A)

Schematy czekania/wycofywania na przykładzie

Rozważmy transakcje T_1 i T_2 takie, że T_1 ma wyższy priorytet niż T_2 .

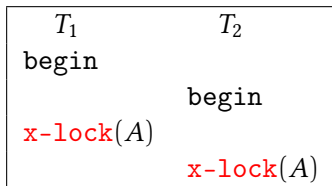


schemat wait-die:

$\rightsquigarrow T_1$ czeka.

schemat wound-wait:

$\rightsquigarrow T_2$ zostaje wycofana.



schemat wait-die:

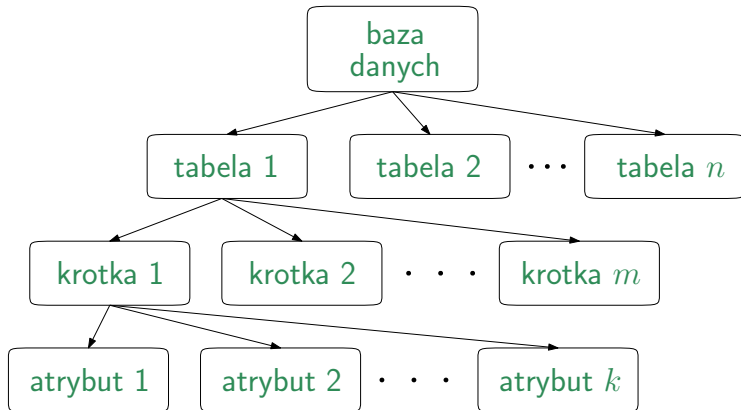
$\rightsquigarrow T_2$ zostaje wycofana.

schemat wound-wait:

$\rightsquigarrow T_2$ czeka.

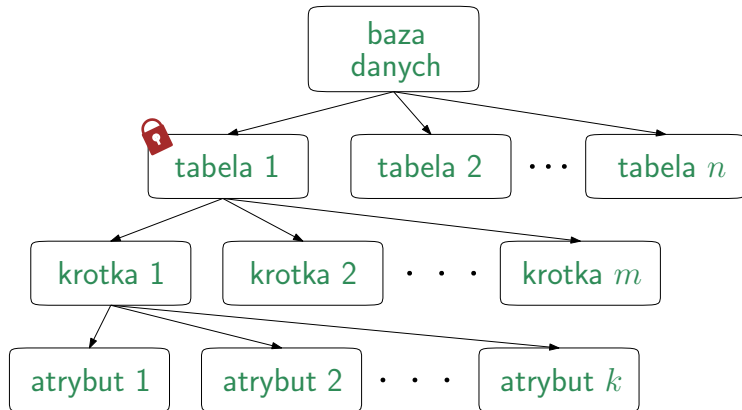
Blokujemy... ale co?

Blokady mogą być założone na różne warstwy w hierarchii bazy danych: na atrybuty, krotki, strony w plikach, tabele, całą bazę.



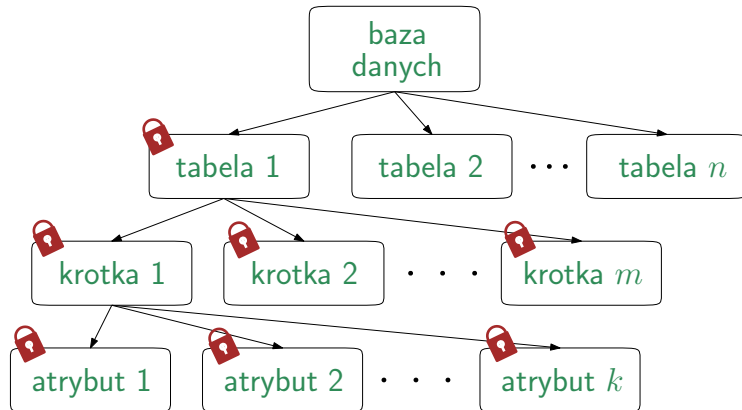
Blokujemy... ale co?

Blokady mogą być założone na różne warstwy w hierarchii bazy danych: na atrybuty, krotki, strony w plikach, tabele, całą bazę.



Blokujemy... ale co?

Blokady mogą być założone na różne warstwy w hierarchii bazy danych: na atrybuty, krotki, strony w plikach, tabele, całą bazę.



Blokady intencyjne

Blokada intencyjna pozwala nałożyć blokadę na wyższy węzeł w drzewie hierarchii. Dzięki temu bez sprawdzania niższych warstw potrafimy stwierdzić, czy jakiś obiekt w niższej warstwie ma założoną blokadę (dzieloną lub wyłączną).

Jeżeli węzeł otrzymuje blokadę intencyjną, to właściwa blokada jest zakładana w niższej warstwie.

Blokady intencyjne

Blokada intencyjna pozwala nałożyć blokadę na wyższy węzeł w drzewie hierarchii. Dzięki temu bez sprawdzania niższych warstw potrafimy stwierdzić, czy jakiś obiekt w niższej warstwie ma założoną blokadę (dzieloną lub wyłączną).

Jeżeli węzeł otrzymuje blokadę intencyjną, to właściwa blokada jest zakładana w niższej warstwie.

Wyróżniamy trzy rodzaje blokad intencyjnych:

- IS – **intencyjna dzielona**, wskazująca na istnienie blokad dzielonych w którejs z niższych warstw,

Blokady intencyjne

Blokada intencyjna pozwala nałożyć blokadę na wyższy węzeł w drzewie hierarchii. Dzięki temu bez sprawdzania niższych warstw potrafimy stwierdzić, czy jakiś obiekt w niższej warstwie ma założoną blokadę (dzieloną lub wyłączną).

Jeżeli węzeł otrzymuje blokadę intencyjną, to właściwa blokada jest zakładana w niższej warstwie.

Wyróżniamy trzy rodzaje blokad intencyjnych:

- IS – **intencyjna dzielona**, wskazująca na istnienie blokad dzielonych w którejś z niższych warstw,
- IX – **intencyjna wyłączna**, wskazująca na istnienie blokad wyłącznych (i być może również dzielonych) w którejś z niższych warstw,

Blokady intencyjne

Blokada intencyjna pozwala nałożyć blokadę na wyższy węzeł w drzewie hierarchii. Dzięki temu bez sprawdzania niższych warstw potrafimy stwierdzić, czy jakiś obiekt w niższej warstwie ma założoną blokadę (dzieloną lub wyłączną).

Jeżeli węzeł otrzymuje blokadę intencyjną, to właściwa blokada jest zakładana w niższej warstwie.

Wyróżniamy trzy rodzaje blokad intencyjnych:

- IS – **intencyjna dzielona**, wskazująca na istnienie blokad dzielonych w którejś z niższych warstw,
- IX – **intencyjna wyłączna**, wskazująca na istnienie blokad wyłącznych (i być może również dzielonych) w którejś z niższych warstw,
- SIX – **dzielona i intencyjnie wyłączna**, wskazująca na istnienie blokad wyłącznych w którejś z niższych warstw i nadająca transakcji współdzielony dostęp do całego poddrzewa.

Macierz kompatybilności

Poniższa tabela odwzorowuje zależności między blokadami:

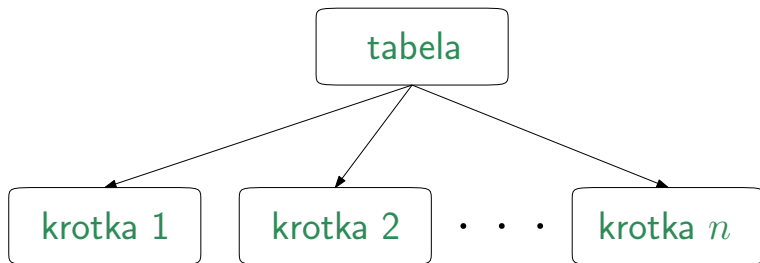
- IS – intencyjna dzielona,
- IX – intencyjna wyłączna,
- S – dzielona,
- SIX – dzielona i intencyjnie wyłączna,
- X – wyłączna.

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	✗
IX	✓	✓	✗	✗	✗
S	✓	✗	✓	✗	✗
SIX	✓	✗	✗	✗	✗
X	✗	✗	✗	✗	✗

Więcej blokad: przykłady

Rozważmy trzy transakcje wykonywane na pewnej tabeli:

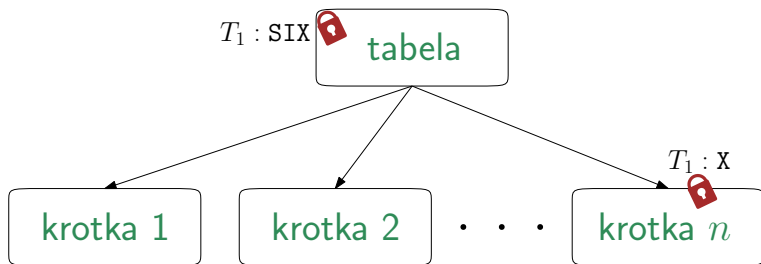
- T_1 czyta wszystkie krotki i modyfikuje jedną z nich,
- T_2 czyta jedną krotkę,
- T_3 czyta wszystkie krotki.



Więcej blokad: przykłady

Rozważmy trzy transakcje wykonywane na pewnej tabeli:

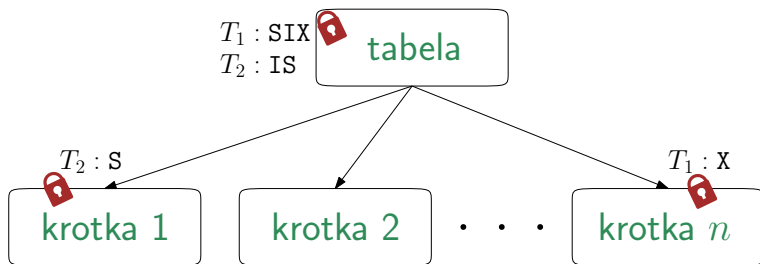
- T_1 czyta wszystkie krotki i modyfikuje jedną z nich,
- T_2 czyta jedną krotkę,
- T_3 czyta wszystkie krotki.



Więcej blokad: przykłady

Rozważmy trzy transakcje wykonywane na pewnej tabeli:

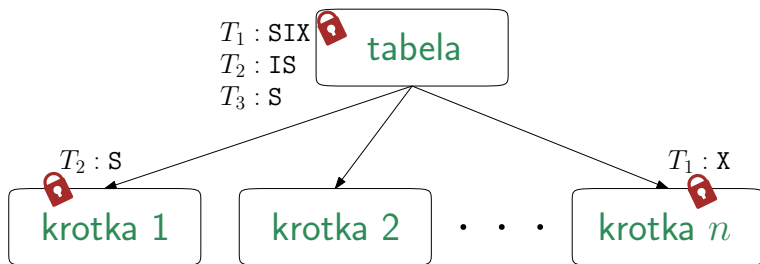
- T_1 czyta wszystkie krotki i modyfikuje jedną z nich,
- T_2 czyta jedną krotkę,
- T_3 czyta wszystkie krotki.



Więcej blokad: przykłady

Rozważmy trzy transakcje wykonywane na pewnej tabeli:

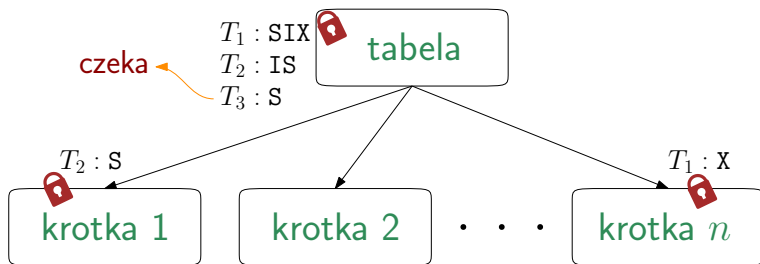
- T_1 czyta wszystkie krotki i modyfikuje jedną z nich,
- T_2 czyta jedną krotkę,
- T_3 czyta wszystkie krotki.



Więcej blokad: przykłady

Rozważmy trzy transakcje wykonywane na pewnej tabeli:

- T_1 czyta wszystkie krotki i modyfikuje jedną z nich,
- T_2 czyta jedną krotkę,
- T_3 czyta wszystkie krotki.



Ziarnistość blokad

Hierarchiczne blokowanie jest wykorzystywane w praktyce, gdyż dzięki niemu każda transakcja blokuje tylko niewielką część danych. Pozwala to na zwiększenie współbieżności.

Jeśli transakcja żąda wielu blokad na niższej warstwie, bardziej opłacalne może się okazać przyznanie jej blokady dla całego poddrzewa. To z kolei zmniejsza liczbę żądań przesyłanych do menadżera blokad.

Ziarnistość blokad

Hierarchiczne blokowanie jest wykorzystywane w praktyce, gdyż dzięki niemu każda transakcja blokuje tylko niewielką część danych. Pozwala to na zwiększenie współbieżności.

Jeśli transakcja żąda wielu blokad na niższej warstwie, bardziej opłacalne może się okazać przyznanie jej blokady dla całego poddrzewa. To z kolei zmniejsza liczbę żądań przesyłanych do menadżera blokad.

Zazwyczaj nie przyznaje się blokad ręcznie, jednak niektóre systemy bazodanowe to umożliwiają (choć nie jest to w standardzie SQL).

Ziarnistość blokad

Hierarchiczne blokowanie jest wykorzystywane w praktyce, gdyż dzięki niemu każda transakcja blokuje tylko niewielką część danych. Pozwala to na zwiększenie współbieżności.

Jeśli transakcja żąda wielu blokad na niższej warstwie, bardziej opłacalne może się okazać przyznanie jej blokady dla całego poddrzewa. To z kolei zmniejsza liczbę żądań przesyłanych do menadżera blokad.

Zazwyczaj nie przyznaje się blokad ręcznie, jednak niektóre systemy bazodanowe to umożliwiają (choć nie jest to w standardzie SQL).

Przykład. PostgreSQL dopuszcza różne blokady, w tym między innymi: ACCESS SHARE, SHARE UPDATE EXCLUSIVE, SHARE, EXCLUSIVE, ACCESS EXCLUSIVE. Blokadę zakłada się poleceniem

```
LOCK TABLE tabela IN typ_blokady MODE;
```

To jeszcze nie wszystko

Jeśli transakcje zawierają tylko operacje czytania i modyfikowania istniejących krotek, to odpowiednie stosowanie blokad zapewni nam ich poprawną realizację.

Jeśli jednak zawierają również operacje wstawiania i usuwania, to pojawia się nowy problem.

To jeszcze nie wszystko

Jeśli transakcje zawierają tylko operacje czytania i modyfikowania istniejących krotek, to odpowiednie stosowanie blokad zapewni nam ich poprawną realizację.

Jeśli jednak zawierają również operacje wstawiania i usuwania, to pojawia się nowy problem.

T_1	T_2
begin	begin
SELECT AVG(ocena)	
FROM studenci	
	INSERT INTO studenci
	VALUES ('Jan', 3)
	commit
SELECT AVG(ocena)	
FROM studenci	
commit	

Uwaga, straszy

W przykładzie blokowanie było realizowane na poziomie krotek.

Transakcja T_1 nie widzi krotki wprowadzanej przez transakcję T_2 i nie mogła założyć blokady odczytu na tej krotce.

Uwaga, straszy

W przykładzie blokowanie było realizowane na poziomie krotek.

Transakcja T_1 nie widzi krotki wprowadzanej przez transakcję T_2 i nie mogła założyć blokady odczytu na tej krotce.

Takie krotki nazywane są **fantomami** lub **duchami**.

Ich pojawienie się sprawia, że realizacja transakcji może być niepoprawna, a jednak dozwolona przez algorytm ścisłego blokowania dwufazowego.

Nie istnieje żaden mechanizm blokowania realizowany na poziomie krotek, który gwarantowałby rozwiązanie problemu fantomów. Rozwiązanie tego problemu wymaga wprowadzenia blokad hierarchicznych.

Poziomy izolacji w bazach danych

Standard SQL definiuje następujące poziomy izolacji transakcji:

- poziom 0 (**read uncommitted**):
dozwolone są odczyty niezatwierdzonych danych,

Poziomy izolacji w bazach danych

Standard SQL definiuje następujące poziomy izolacji transakcji:

- poziom 0 (**read uncommitted**):
dozwolone są odczyty niezatwierdzonych danych,
- poziom 1 (**read committed**):
tylko zatwierdzone zmiany mogą zostać odczytane, jednak niepowtarzalne odczyty są dopuszczalne,

Poziomy izolacji w bazach danych

Standard SQL definiuje następujące poziomy izolacji transakcji:

- poziom 0 (**read uncommitted**):
dozwolone są odczyty niezatwierdzonych danych,
- poziom 1 (**read committed**):
tylko zatwierdzone zmiany mogą zostać odczytane, jednak niepowtarzalne odczyty są dopuszczalne,
- poziom 2 (**repeatable read**):
zmienione dane mogą być odczytane tylko po zatwierdzeniu, pomiędzy dwoma odczytami tej samej danej przez dwie transakcje żadna inna transakcja nie może tej danej zmieniać, mogą się jednak pojawić fantomy,

Poziomy izolacji w bazach danych

Standard SQL definiuje następujące poziomy izolacji transakcji:

- poziom 0 (**read uncommitted**):
dozwolone są odczyty niezatwierdzonych danych,
- poziom 1 (**read committed**):
tylko zatwierdzone zmiany mogą zostać odczytane, jednak niepowtarzalne odczyty są dopuszczalne,
- poziom 2 (**repeatable read**):
zmienione dane mogą być odczytane tylko po zatwierdzeniu, pomiędzy dwoma odczytami tej samej danej przez dwie transakcje żadna inna transakcja nie może tej danej zmieniać, mogą się jednak pojawić fantomy,
- poziom 3 (**serializable**):
realizacja transakcji jest szeregowalna.

Poziomy izolacji

Im niższy poziom, tym większa współbieżność.

Im wyższy poziom, tym większy stopień poprawności.

Dopuszczalność anomalii na różnych poziomach izolacji:

	brudne odczyty	niepowtarzalne odczyty	fantomy
0: read uncommitted	tak	tak	tak
1: read committed	nie	tak	tak
2: repeatable read	nie	nie	tak
3: serializable	nie	nie	nie

Poziomy izolacji

Im niższy poziom, tym większa współbieżność.

Im wyższy poziom, tym większy stopień poprawności.

Dopuszczalność anomalii na różnych poziomach izolacji:

	brudne odczyty	niewtarzalne odczyty	fantomy
0: read uncommitted	tak	tak	tak
1: read committed	nie	tak	tak
2: repeatable read	nie	nie	tak
3: serializable	nie	nie	nie

```
BEGIN TRANSACTION ISOLATION LEVEL poziom;
```